

An Energy-Efficient Smart Metering System Using LoRa Network

A MAJOR PROJECT REPORT

Submitted by

KUDITHIPUDI DHANVEER(RA1911004020132)

SIRIMARAM MAHIDHAR(RA1911004020133)

VADDEPALLY PUNITH KUMAR(RA1911004020147)

Under the Guidance of

Dr D. MANOJ KUMAR

(Assistant Professor)

**DEPARTMENT OF ELECTRONICS AND
COMMUNICATION ENGINEERING
FACULTY OF ENGINEERING & TECHNOLOGY**



SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

RAMAPURAM, Chennai – 600 089

MAY-2023

SRM INSTITUTE OF SCIENCE & TECHNOLOGY

(Deemed To be University, Under Section 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this major project report titled “**AN ENERGY EFFICIENT SMART METERING SYSTEM USING LoRa NETWORK**” is the bonafide work of “**KUDITHIPUDI DHANVEER (RA1911004020132), SIRIMARAM MAHIDHAR (RA1911004020133), VADDEPALLY PUNITH KUMAR (RA1911004020147)**” who carried out the major project work under my supervision. Certified further, that to the best of my knowledge, the work reported herein does not form any other project report or dissertation based on which a degree or award was conferred on an earlier occasion on this or any other candidate.

GUIDE SIGNATURE

Dr D. MANOJ KUMAR

Assistant Professor

Department of Electronics and
Communication Engineering

SRM Institute of Science and
Technology

Ramapuram

Chennai-600089

SIGNATURE

Dr. PHANI KUMAR POLASI

Professor and Head

Department of Electronics and
Communication Engineering

SRM Institute of Science and
Technology

Ramapuram

Chennai-600089

Signature of the Examiner - I

Signature of the Examiner- II

ACKNOWLEDGMENT

We are placed regarding our deep sense of gratitude to our beloved Chancellor to **Dr T.R. PACHAMUTHU**, for providing us with the requisite infrastructure throughout the course. We take the opportunity to extend our hearty thanks to our Chairman, **Dr R. SHIVAKUMAR** for his constant encouragement. We take the opportunity to extend our hearty thanks to our Dean, **Dr. M. MURALI KRISHNA** for his constant encouragement. We convey our sincere thanks to our Vice Principal (Academic), **Dr. G. PRABHAKARAN** for his interest and support throughout this project. We take the privilege to extend our hearty thanks to the Head of the Department **Dr PHANI KUMAR POLASI**, for his suggestions, support, and encouragement towards the completion of the project with perfection. We thank our faculty coordinator **Dr D. MANOJ KUMAR**, Assistant Professor – Department of Electronics and Communication Engineering for his sustained encouragement, consecutive criticisms, and Support throughout this projectwork.

ABSTRACT

In reaction to the Paris Climate Conference (COP21), several traditional fossil fuel-based power generation plants have been transformed into renewable energy plants on a global scale. Due to unpredictable environmental variables, such as the weather, renewable energy sources have fluctuating production and are challenging to run on a set timetable. By gathering, analyzing, and acting on data on the state of continuous power generation, the power generation system may be controlled more steadily. The accumulated data can be used to forecast future energy production and facilitate necessary maintenance. The flexibility and dependability of the grid have increased due to this improved stability. This report presents an implementation strategy for building an effective energy monitoring system in this study using open IoT hardware and software platforms. Many sensors are used to monitor a renewable energy-producing plant. In this case, the energy produced is one of the main characteristics that need to be tracked; it may be estimated by measuring current and voltage. It is simple to compute energy produced using current and voltage values. Low-cost solutions are used to implement LoRa, which supports low-power long-distance networks.

Index Terms — Internet of Things, Arduino UNO, NodeMCU, LoRa, PV Panel.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ACKNOWLEDGEMENT ABSTRACT LIST OF FIGURES	3 4 6-7
1	INTRODUCTION	8
2	LITERATURE REVIEW	9-11
3	EXISTING & PROPOSED SYSTEM 3.1-Existing System 3.2-Proposed System Hardware Requirements Software Requirements	12-52 12 12-44 44-52
4	METHODOLOGY 4.1 -Flow Diagram	53-54
5	RESULTS 5.1-Results	55-56
6	CONCLUSION & FUTURE SCOPE 6.1 Conclusion 6.2 Future Scope	57
	REFERENCES	58-59
	APPENDIX	60-64

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1	Arduino Uno	13
2	Output Signals	19
3	NodeMCU	24
4	NodeMCU Pins	26
5	Pin Definitions of SIDO'S	27
6	Pin Definitions of SPI'S	28
7	Pin Definitions of I2C	28
8	Pin Definitions of UART	29
9	Pin Definitions of PWM	29
10	WIFI Module	31
11	LoRa	32
12	Working of LoRa	33
13	LoRa Complements WIFI, Bluetooth & Cellular	34
14	LoRa Availability	34
15	LoRa Network Across the Globe	35
16	A Rapidly Scaling Ecosystem	35

17	Simplified Go-To-Market	36
18	LoRa Architecture	37
19	Voltage Sensor	38
20	Voltage Source Whose Voltage to be Measured	39
21	Capacitive Type Voltage Sensor and Resistive Type Voltage Sensor	40
22	Capacitive Type Voltage Sensor	40
23	Voltage Resistive Sensor	42
24	Voltage Resistive Sensor with a Resistance Bridge	42
25	PV Panel	44
26	Language Support	51
27	Transmitter (Tx)-Sensor Node	53
28	Receiver (Rx)-Data Node	54
29	Circuit Diagram	55
30	The output of TRANSMITTER (Tx) - SENSOR NODE	55
31	The output of RECEIVER (Rx)- DATA NODE	56
32	Output	56

CHAPTER-1

INTRODUCTION

The networking of intelligent objects to perform shared tasks is known as the Internet of Things (IoT). The goal of IoT is to enhance individual experiences and livelihood. Smart meters provide remote energy use monitoring, which is crucial for both users and the operator. One of the potential applications of IoT is this. Through the internet, these items can interact with people and other sensing devices. Data is transferred from one device to another through communication between the digital and electrical technologies employed in smart meters. The public now has access to the smart meter as an alternative to traditional power meters. Metering staff from the energy delivery providers must inspect each conventional power meter individually. On the other hand, smart meters that support the Internet of Things (IoT) give customers accurate automatic meters as well as the ability to remotely monitor and regulate their energy use. Smart meters can encourage significant and reasonably priced customer participation in the utility business and raise end-user involvement in energy conservation. A few projects aimed at reducing climate change and maintaining energy security can also be supported by smart meters.

CHAPTER-2

LITERATURE SURVEY

Otisitswe Kebotogetse, et.al “Sophisticated Metering Infrastructure: A Hidden Based Method to Secure Communication” Published in 2022. The Automated Metering Infrastructure (AMI), a crucial component of the smart electricity system, is in charge of monitoring client power use. Smart meters, which are crucial for simplifying the interaction between the service provider and consumers, are one of the elements of the AMI subsystem. Actual readings of the customers’ power usage can be sent to the utility through smart meters. For the secure transmission of readings, the security of this communication route against intruders is still essential.

Preti Kumari, et.al “A LoRa network-based edge computing system for energy-efficient smart meters” Published in 2021. This research suggests an Edge computing in Long Range (LoRa)-based smart metering system that is energy-efficient. To process and send the processed smart meter readings, respectively, an Edge device and a LoRa node are attached to a smart meter that is connected to all of the household's appliances. An energy multivariate time series can be used to describe the appliances' energy usage.

YAHIA BAASHAR, et.al “An Empirical Investigation of IoT-Based Smart Meter Usage in Malaysia: The Evolution of Energy-Saving Data and Environmental Awareness" released in 2019. The rising demand for electricity today necessitates finding solutions that better balance the supply and demand of energy. Among power providers, innovative technical solutions like smart metering programs are becoming more popular. Despite having many advantages, smart meters, a component of the Internet of Things (IoT), are still having trouble gaining public acceptability due to a lack of customer understanding of electricity savings and environmental concerns.

FATIH ÜNAL, et.al “Big Data Detection of Fake Information Manipulation Threats in Smart Meters, 2021. Other Advanced Metering Systems (AMI) and smart meters have made energy supplies more readily accessible to users. Smart grids use a variety of interfaces, including social, physical, and cyber interfaces, to control and operate smart meters. Recently, several non-technical losses (NTLs) in smart grids across the globe have been documented. False data injections (FDIs), in part, are to blame for them. To guarantee a consistent supply of power to consumers, it is essential to have a secure communication channel and secure AMIs.

Luke Kane, et.al, “Published in 2022: "LoRa 2.4 gigahertz Home Automation Network Construction and Authentication Method". Energy providers and consumers can both benefit from the IoT-based smart grid's advanced metering features, better dependability, and management. Security is becoming a bigger issue as smart homes and cities proliferate because data networks are increasingly running alongside electrical networks. In the smart home, it is crucial to apply strong security procedures.

Fangxing Liu, et.al. “2020 publication of "Remote Identification of Inoperative Smart Meters in an Edge-Based Computing Environment". An example of an edge gadget that captures and analyses information about energy is the smart meter. The data security-physical connection that exists between the smart power grid and the physical system of the internet may be strengthened using data from smart meters. Millions of smart meters have been installed worldwide, and the high rate of meter defect detection is a big problem. Big-scale meters are in high demand, and on-site verification is a costly process that cannot meet that demand.

Zhuoqun Xia, et.al “Meter-reading System Using Secure Session Key Management Using LoRa Technology” Published in 2016. The long-range (LoRa) technology for wireless communication may be used to deliver negligible power, negligible rate, and long-range communication. It may be applied in a variety of ways, including smart electricity meters and industrial surveillance. LoRa technology may introduce some new security flaws, though. So that the LoRa-based meter reading technology is protected, this paper advises using an upgraded session key management mechanism.

Jin Young Kim, et.al “Smart Energy Theft Detection for Non-Technical Losses in Smart Grid with Intermediate Surveillance Meter” Published in 2019. This paper suggests an interim monitor meter (IMM)-based distribution of power network of things framework with the idea of unit networks (UNs) to accurately identify the not technically complex loss (NTL) of meter trying to control/malfunctioning and getting around at the same time and to investigate the power flow in detail. After describing the hardware architecture of IMMs, the linear system of equations (LSE) generated by the energy balance analysis utilizing IMMs and the collection device is subsequently solved using an NTL detection approach.

Mihai Sanduleac, et.al.” Utilizing High Transmission Rate Data from Smart Meters, Improved Grid Surveillance and Services for Energy Users” Published in 2022. The fluctuating nature of the electricity transmission systems is something that grid controllers are highly interested in recording, particularly when there is a considerable penetration of

renewable energy sources. Lack of metering equipment makes it difficult to properly audit distribution power networks, especially those downstream of medium-voltage substations. As a result, it is suggested that data collected at noticeably different reporting rates be merged to improve the system's situational awareness with an influence on the tracking tool's effectiveness that is not negligible.

José Luis Gallardo, et.al.” Network Planning Using Clustering Algorithms for Improved Metering Infrastructure in the Smart Grid Published in 2021. Modern electric grids are replacing outdated electrical distribution systems with ones that can handle a variety of applications, including distributed energy sources, Electric transportation, and advanced metering infrastructure (AMI). These applications include data delivery from end users to energy providers, trustworthy continuous surveillance, remote management of voltage history and electrical performance data, and more. AMI is playing a significant part in all these applications. The utility control center, communication networks, data aggregation points (DAPs), and smart meters.

CHAPTER-3

EXISTING & PROPOSED SYSTEM

3.1 EXISTING SYSTEM

Unmonitored power generation systems
Local monitoring systems

3.2 PROPOSED SYSTEM

A monitoring mechanism for systems that generate renewable energy in this system. The many nodes in this system are connected using LoRa communication. The power generated is monitored using voltage as well as current sensors, and the information is transferred to a distant collecting node for additional analysis. IoT makes remote data access possible in this case. In this case, remote data access is made possible through IoT. The Arduino platform is being used to create the project.

Hardware Requirements

Arduino Uno

Based on the Microchip ATmega328P CPU, the Arduino Uno, or Uno, a microcontroller is free and open-source and was developed by Arduino. cc. On the circuit board, there are groups containing both digital and analogue I/O pins that may be used to link to different expansion chips and other circuits. The chip's fourteen pins are digital, and six pins are analogue and may be programmed using the Integrated Development Environment (IDE) through a B-type USB connector. It may be powered by an external 9-volt battery or USB port in addition to the 7 - 20-volt range. Both the one called the Arduino Nano and the Leonardo are equivalent. A series of directives will be sent to the board's microcontroller, instructing it on what to perform. This is accomplished by using the Wiring language of programming and the Arduino Software (Integrated Development Environment), which depends on Processing.



Figure 1: Arduino Uno

ARDUINO:

setup (): A component found in each Arduino code. Run once before using the loop () method. Frequently used to set the I/O pin type. The setup () method syntax is as follows:

```
void setup () {  
    //code goes here  
}
```

loop (): A component found in every Arduino program. This exact code appears frequently. The loop () is where (nearly) all the action takes place. Variable declaration and setup () are the only exceptions to this rule. The "forever ()" loop, which is used by ModKit, executes over Serial. The loop () function is formatted as follows:

```
void loop () {  
    //code goes here  
}
```

input: A pin mode that will take data.

output: A pin mode that sends information.

HIGH: A signal with 5Volts for Uno is present electrically. Moreover, True or ON in Boolean logic.

LOW: A zero-voltage electrical signal. Additionally, in Boolean logic, OFF or False.

digitalRead: Check the HIGH or LOW reading of an input-designated pin.

digitalWrite: An electrical pin that has been programmed to produce a Higher or Lower value should be provided.

analogRead: A number that falls between 0 and 1023 or a value therein is requested. Because of this, readings from analogue sensors or interfaces with many states can be obtained.

analogWrite: A value between 0 (LOW) and 255 (HIGH) or a value therein should be assigned. This will enable you to switch the output from HIGH or LOW to a PWM value.

PWM: Pulse-width modulation, or PWM, is a technique for transmitting a signal that is analogue via a digital pin. A number that falls within the range of 0 and 255. makes use of **analogue Writing**.

The microcontroller in the boards below is the same one; the packaging is different. The Arduino Mini is essentially a smaller container without the USB, Barrel Jack, and Power Outs, but the Lilypad is intended for use with conductive thread rather than wire.

It depends on what you want to do with it. We shall discuss both voltage dividers' two distinct functions.

If you want to use the voltage divider as a sensor reading device, to understand the signal, you must be informed of the maximum voltage that is permitted through the analogue I/P you're utilizing. The Arduino does this using 5V. The greatest value that V_{out} may have been previously known. The voltage present on the circuit before the first resistor is referred to as V_{in} . The datasheet for your sensor should provide the maximum voltage that it can produce; this is the highest voltage that your sensor will permit to pass when it is operating with the circuit's input voltage. The value of the second resistor is the only variable that is left at this point. By solving for R_2 , you may determine each part of the voltage divider. Because the smaller resistor would just offer a weaker signal that might be picked up by our analogue inputs, R_1 's maximum value must be ascertained.

Like reading a sensor, generating an analogue reference requires figuring out the V_{out} quantity you wish to utilize as the analogue Standard.

Electrical signals that are either digital or analogue can be used with the Arduino. It is essential to comprehend the variations among these two signal kinds and how to change the data that these signals convey.

DIGITAL:

A binary-coded electrical signal can either be the presence or absence of current, a mixture of two distinct voltages (high or low), or a series of brief pulses at a specific frequency.

Robots, computers, and circuits all use digital technology whereas humans experience the world as an analogue. A signal with just two states is said to be digital. These states can fluctuate concerning the signal being transmitted, but they are never in between; they are either ON or OFF.

All signals in the Arduino universe, except for analogue input, are digital. Despite the off position or LOW signal, which is always 0V, according to the voltage generated by the Arduino, the ON state (HIGH) of the electrical signal being formed will be equivalent to the system voltage. This is a clever way of indicating that the Max signals on the 3.3 Arduino voltage will be just below 3.3V, and the HIGH signals on a 5-volt Arduino will be slightly below 5V.

Digital pins 0 through 13 are used by the Arduino to receive or deliver digital signals. Your Analogue In pins can be configured to function as Digital pins as well. Use the following command to convert Analogue In pins to Digital Pins:

➤ `pinMode (pinNumber, value);`

In this case, the pin Number represents an analogue pin (A0 to A5), while the value is either an input or an output. Use the identical command to configure digital pins instead of an analogue pin, referencing a digital pin for the pin Number. In reality, you just need to change digital pins to output in `pinMode` because they to being input. Apply the following command to read these pins:

➤ `digitalRead(pinNumber);`

pin Number here denotes the pin number of the digital component. Either a HIGH value or a LOW value will be returned by the digital Read instruction. Use the guidelines below to send a signal that is digital to a pin:

➤ `digitalWrite (pinNumber, value);`

The value of the pin Number (which is the pin's identification number) is either HIGH or LOW.

Pulse Width Modulation (PWM), a signal that the Arduino may emit, which replicates an analogue signal, is a signal that is digital. PWM functionality is available on electronic PINs 3, 5, 6, 9, 10, and 11. This is what the command "output a PWM signal" does.

➤ `analogWrite (pinNumber, value);`

AnalogWrite (pin Number, value) is used if the value of pin Number, a digital pin with PWM capabilities, falls between 0 and 255. Look at the S.I.K. circuit 12 or the PWM spreadsheets for further information on PWM.

THINGS TO REMEMBER ABOUT DIGITAL:

- Digital pins are used for digital input and output, nonetheless, digital can also use analogue pins.
- To acquire a digital signal, use the digital Read (pin Number) function;
- To send a digital signal, use the digital Write (pin Number, value) method.
- The I/O of digital devices can only be HIGH or LOW.

Electrical signals that are either digital or analogue can be used with the Arduino. It is essential to comprehend the variations compared to these two signals kinds and how to modify the information that these signals transmit.

ANALOG:

Humans have analogue perceptions of the world. Our senses are constantly receiving information from everything we see and hear. The temperatures experienced fluctuate within our ranges of comfortable temperatures; they are never completely hot or cold. (And what exactly are we performing there if they're above or below the range of temperatures we are allowed?) The uninterrupted stream of analogue data is its defining feature. The analogue concept's complementary idea, digital information, calculates analogue using data just 1's and 0's.

A single signal in the Arduino world that may be ON(HIGH), OFF(LOW), or in any other position among both is analogue. This suggests that barring interference with the Analogue Reference pin, the range of the voltage value of an analogue signal is from 0V to 5V. Analogue signals can be used for I/O from devices that work at proportions in addition to switching between modes (on and off). Using an example of the electrical voltage output

provided to these pins and a comparison to the default voltage signal (5V), Arduino does this. The Arduino then assigns the signal a value that ranges from 0 (%) to 1023 (100%) after comparing the voltage of the analogue signal to the analogue reference signal. Then, you may utilize this number in computations and sketches using Arduino's digital system.

When receiving analogue input, Arduino uses analogue pins 0 through 5. These pins can be used for analogue input and are intended for use with devices that output analogue data. There is no preparation required, the command to read them is:

➤ `analogRead(pinNumber);`

pin Number serves as an analogue. The outcome of the analogue Read command is a number between 0 and 1023 in the pin to which the analogue component is connected., or thereabouts.

Pulse Width Modulation (PWM), a signal that Arduino may emit, which replicates an analogue signal, is a digital signal. PWM is supported on Digital Pins #5, #3, #9, #6, #10, and #11. Use the directive to generate a PWM signal,

➤ `analogWrite (pinNumber, value);`

which value signifies a value between 0 (zero) and 255 (100%), and pin Number refers to a digital pin with PWM capability. The PWM pins of the Arduino UNO are identified by symbols. See the PWM worksheets or S.I.K. circuit 12 for additional details on PWM.

THINGS TO REMEMBER ABOUT ANALOG:

- Analogue Input makes use of the Analogue Output and employs the PWM pins in terms of pins.
- Use the command `analogRead (pin Number)` to obtain an analogue signal.
- By utilizing the `analogWrite (pin Number, value)` function, a PWM signal can be sent.
- The analogue input values range from 0 to 1023 because it employs 10 bits, or 210.
- Due to the utilization of 8 bits, 28 PWM output values have a range of 0 to 255.

The Arduino uses input and output for each electrical signal. It is essential to comprehend the variations ways to alter the data that these signals communicate, and the differences between these two signal types.

OUTPUT SIGNALS:

The digital output is provided in two different formats:

1. Standard digital output
2. Pulse width modulation (PWM).

The Arduino pins always receive digital output. Only the Digital pins 0 through 13 support output. There is no requirement to utilize the `pinMode` instruction to switch the default state of the Digital pins from Output to Input if the pin is not utilized as an Input in the same sketch. If it becomes required to switch a digital pin from input to output, execute the following command:

➤ `pinMode (pinNumber, OUTPUT);`

The digital pin number is `pin Number`, chosen for the formula's Output Pin Mode (`pin Number, OUTPUT`). Utilize the following command to send a Digital Output signal:

➤ `digitalWrite (pinNumber, value);`

Value is the signal itself, and Pin Number identifies the digital pin that is transmitting it. The output value of a digital signal can either be ON (HIGH) or OFF (LOW).

➤ `analogWrite (pinNumber, value);`

The value is between 0 and 255, where the pin Number is a digital pin that can do PWM. For further information on PWM, see the S.I.K. circuit 12 or the pulse width modulation worksheets.

The user must decide which output signal type is needed, link the necessary components, and then input the right code to maximize these outputs. The output can be received by a wide range of devices.

THINGS TO REMEMBER ABOUT OUTPUT:

- Two different output types exist PWM (Pulse Width Modulation) and conventional digital.
- Digital o/p is always used.
- Use `analogWrite (pin Number, value);` or `digitalWrite (pin Number, value);` to transmit an output signal for analogue or digital output, respectively.

- The pinMode command allows you to change the output pin mode: pinMode (pinNumber, OUTPUT);
- A typical digital device's output is either HIGH or LOW all the time.
- The PWM output can be set between 0 and 255.

Input and output are used by the Arduino for each and every electrical signal. It is essential to comprehend the variations between these two signal kinds and how to change the data that these signals convey.

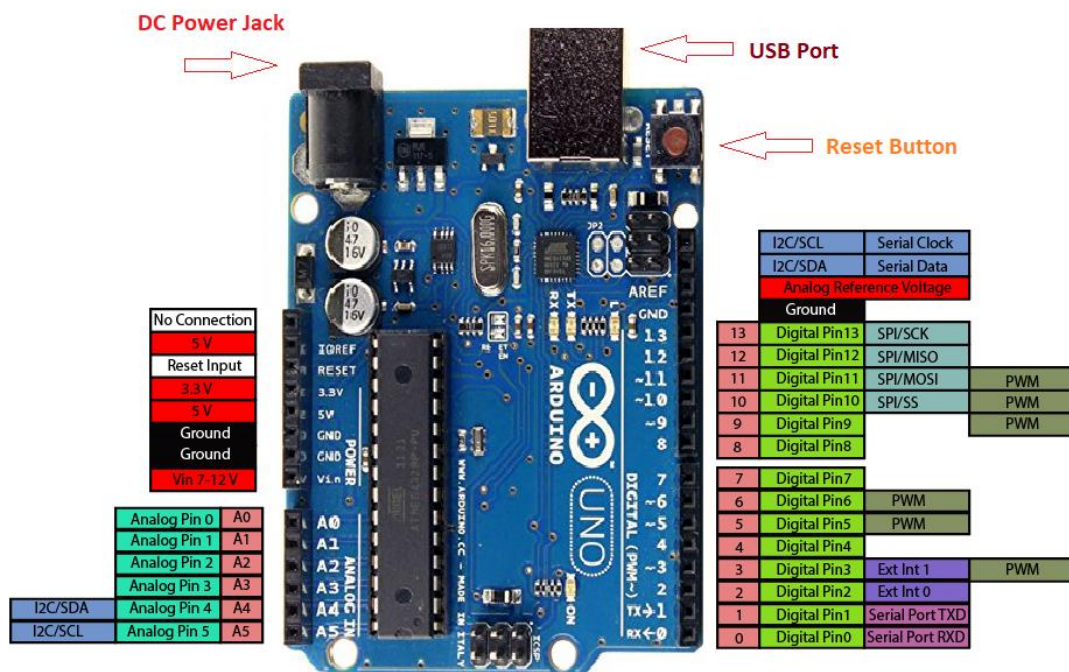


Figure 2: Output Signals

INPUT SIGNALS:

The Parallel Analogue input enters your Arduino at pins #0 through #5. These signals are generated via analogue sensors and interfaces. These analogue sensors and devices express their information using voltage levels rather than a simple yes or no. As a result, these gadgets are unable to accept input from a digital pin. Analogue input pins can only receive analogue signals. The Analogue Input pins can only be read; hence the setup () method does not require a command to ready them for i/p. Use the following command to read the analogue input pins:

- AnalogRead (pinNumber):

where pin Number is the pin number for the analogue input. A reading from an analogue input between 0 and 1023 will be returned by this function. A voltage reading of 1023 is equal to 5 Volts, while a value of 0 is equal to 0 Volts. The analogue sensors and interfaces emit these voltage values. Use the Aref pin to adjust the voltage that 1023 equates to if your analogue input could be higher than $V_{cc} + 5V$. The maximum voltage parameter that your analogue input ports can read is controlled by this pin. The Aref pin's default value is 5V.

Any of the digital pins 0 through 13 can be used to connect a digital input to your Arduino. Input signals for digital devices can be HIGHEST (on, 5VOLT) or LOWEST (off, 0VOLT). And The Digital pins on the Arduino can be used as inputs or outputs, thus you must set them up in your setup () method to use inputs. The command to do this is:

➤ `pinMode (pinNumber, INPUT);`

The pin Number parameter of the setup () procedure, which is surrounded by curly brackets, which you wish to utilize as a source of I/P represents the Digital pin number. PinMode can be altered in the loop () function if you need to alternatively switch one of the pins between I/P and O/P. PinMode is normally supplied in the setup () function & left untouched in the loop () function. To read the digital pins set up as inputs, use the following command:

➤ `digitalRead(pinNumber);`

where pin Number is the pin number for the digital I/p.

To use these signals, the user must first decide what kind of I/P is necessary, connect a hardware component, and then write the necessary code. Although input can come from a wide range of devices, each one will either produce an Analogue or Digital signal.

X1:

- Serial connection DE-9

Used to establish RS-232 connections between computers (or other devices). A serial cable is required, having 2, 3, 4, and 5 pins linked. Only works when JP0 is positioned 2-3.

DC1:

- 2.1 mm power jack

Used to connect an external power supply. centered upbeat. power control Works with regulated DC voltage in the range of +7 to +20 volts (9 to 12 volts is recommended). Either a 9v pin or a 5v pin can be used to connect external power. POWER PINOUT (see).

ICSP:

3pin header, 2x used to create bootloader programmers for Atmega. both sides of the board have the number 1 used to upload Atmega IC sketches without a bootloader (only available in Arduino IDE versions 0011 and 0012) indicating the location of cable pin 1.

JP0:

Jumping three pins The X1 connector is used to enable serial connectivity to/from computers or other devices when this jumper is in positions 2-3. Use this as a starting point. places 1 and 2. Pins 0, which is the Receiver (RX) pin, and 1, which is the transmitter (TX) pin, external pull-down resistors should be turned on. to disable the serial connection. Utilize this just to stop noise on RX, which sporadically prevents the sketch from launching and appears to be incoming data to Atmega. When this jumper is removed, communication via serial is disabled and pins 0 and 1 behave as normal (floating) digital pins. Useful only when a serial connection is not required but more digital pins are required. The use of an exterior pull-down/pull-up resistor is required.

JP4:

double-pin jumper This jumper is used to upload a code to Arduino and reset Atmega devices automatically is enabled while in positions 1-2. When uploading sketches, it is not essential to use the reset button (S1). Make sure the COM Port speed on your computer is set to 19200bps; otherwise, the auto-reset won't function as intended. Disables the automatic reset feature if removed. Very helpful when utilizing sketches that require serial connectivity to avoid unwanted Atmega resets. DTR pulse on serial pin 4 allows auto reset to function. When connecting to X1 (the serial port), Arduino may occasionally detect a DTR pulse. Additionally, certain software may emit a DTR pulse when it opens or closes, which could cause Atmega to reset unintentionally.

S1:

Actual button When the auto-reset feature is off, this button resets the Atmega to enable the Arduino to receive a sketch over the serial port or to resume an uploaded sketch.

LEDS:

LED indicators LEVEL LED The LED illuminates when Arduino is powered by its DC1 pin, +9volt pin, or +5volt pin. The Receiver LED turns on when a computer or another gadget

sends information through a serial interface. The Transmitter LED blinks when data is transmitted through a serial interface to a computer or another gadget. The only component connecting this LED to pin 13 of the digital interface is a current limiter resistor, handy for testing sketches. Blinking is typical during boot loading as well.

POWER PINOUT:

6 pin headers

RST pin:

When connected to GND, causes the Atmega to reset. useful for connecting external reset or shield boards.

NC pin:

The Arduino S3v3 does not have this pin attached. A 3.3 volts pin is present in the same location on the Arduino.

+9v pin:

This pin functions as Vout when Arduino DC1 is supplied with power (by a DC adapter or a battery), supplying an identical voltage (minus 0,7 volts) as is provided on DC1 (see DC1). The capacity of the external power source determines the overall supplied current. When the 0v pin is linked to an external power supply GND, the +9v pin may be utilized as Vin when the Arduino DC1 has no electricity by connecting it to an outside controlled source of power (+7 to +20 volts).

+5v pin:

It may supply +5 volts when used as Vout. +5v.pin The +5v. pin provides +5 volts as a Vout pin when Arduino DC1 is powered DC converter. The voltage Regulator determines how much total current is given (7805 gives up to 1A). Only +5v is affected by this. pin Atmega in/out pins can only deliver a maximum of 40mA per pin. This pin can be used as Vin The +9v. When the Arduino DC1 is not powered, it can be made dormant by attaching the 0v pin to the electricity source Ground (GND) and the +5v pin to a controlled +5v pin. Pin 0V (GND). There is one +9v pin and between +5v and +9v, there are two 0v pins.

0v. pin:

Besides the AREF pin. When Arduino DC1 is powered, 0v.pin supplies 0 volts reference (GND) for +5v. pin and +9v. pin. When DC1 is not powered, and Arduino is powered through +5v.pin or +9v. pin, 0v. pin must be used as a GND reference, connecting it to the external power source GND.

GND pin:

Notice GND at 0 volts.

AREF pin:

The AREF can be external, internal (for Atmega8), 2.56 volts, internal (for Atmega168). or Vcc (the default configuration). When using an external capacitor to isolate the signal from the internal or a Vcc AREF, the performance against noise is improved. When using an external AREF, the external reference voltage is connected to the AREF pin. Before attaching external voltage to AREF, keep in mind that the wiring. The files defuse must be changed and the sketch uploaded again.

TIPS FOR SOFTWARE:

There is a command (-i800) for boot loading an Atmega8 chip using the Arduino 0010 that causes the bootloader to slow by ten minutes. Therefore, if you need to use the bootloader, use the Arduino 0007 IDE or the command line, replacing the "-i800" instruction with the "-F" command. The Arduino 0010 operates wonderfully when it comes to uploading sketches.

ARDUINO S3v3 NEW FEATURES:

Complete compatibility with shield boards (the only Arduino board incompatible with shield boards is version 2 owing to the wrong placement of the ICSP header and tall components);

- an auto-reset function;
- an auto reset enable/disable jumper that prevents unintended resets;
- a reset pin compatible with Arduino;
- an internal LED on pin 13 that has a resistor that limits current;
- inbuilt RX and TX LEDs;
- an adequate current limiter resistor on the power lead (consumption of less than 20mA);

- a jumper that stops "RX floating error" and turns on an external pull-down resistor for RX while also disabling serial transmission. With this functionality, digital pins 0 and 1 can be used as ordinary pins when a serial connection is not necessary.
- The board orientation of all related components (diodes, transistors, LEDs, capacitors) is the same, simplifying installation and minimizing mounting problems;
- There are no wires extending between the pads, there is more room between the wires, the wires are larger, and the pads are larger (better for drilling, etching, and soldering, without any solder bridges, shorts in circuits, or exposed wires that are corroded). To make it easier to employ shield boards, all jumpers are of the right-angle variety.
- An A Vcc LP filter to lower ADC noise levels;

NodeMCU

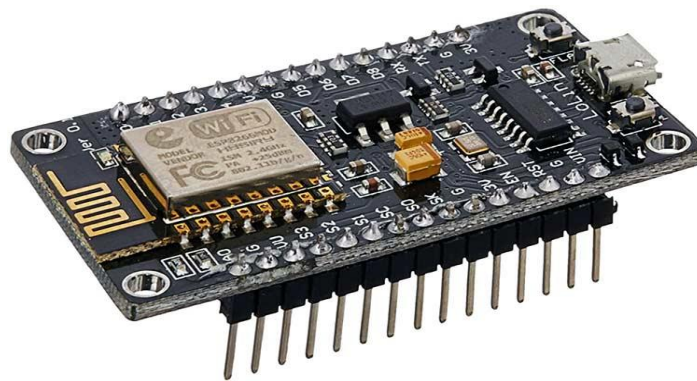


Figure 3: NodeMCU

NodeMCU is a developer board particularly created for IoT applications. Its firmware is powered by Espressif Systems' ESP8266 Wi-Fi SoC, and the motherboard is based on the ESP-12 module from the same company.

NodeMCU Development Board Pinout Configuration

Pin Category	Name	Description
Power	Micro-USB, 3.3V, GND, Vin	Micro-USB: NodeMCU can be powered through the USB port. 3.3V: Regulated 3.3V can be supplied to this pin to power the board GND: Ground pins Vin: External Power Supply
Control Pins	EN, RST	The pin and the button reset the microcontroller
Analog Pin	A0	Used to measure analogue voltage in the range of 0-3.3V
GPIO Pins	GPIO1 to GPIO16	NodeMCU has 16 general-purpose input-output pins on its board
SPI Pins	SD1, CMD, SD0, CLK	NodeMCU has four pins available for SPI communication.
UART Pins	TXD0, RXD0, TXD2, RXD2	NodeMCU has two UART interfaces, UART0 (RXD0 & TXD0) and UART1 (RXD1 & TXD1).

		UART1 is used to upload the firmware/program.
I2C Pins		NodeMCU has I2C functionality support but due to the internal functionality of these pins, you have to find which pin is I2C.

NodeMCU ESP8266 Specifications & Features

A microcontroller having an input voltage range of 7 to 12 volts and an operating voltage of 3.3 volts is the 32-bit RISC CPU Xtensa LX106 from Tensilica.

In addition to 4 MB of flash memory, there are sixteen single analogue input pins (ADC), digital I/o pin,1 UART, 1 SPI, and 1 I2C.

Small Module to Easily Fit Inside Your Internet of Things Project.

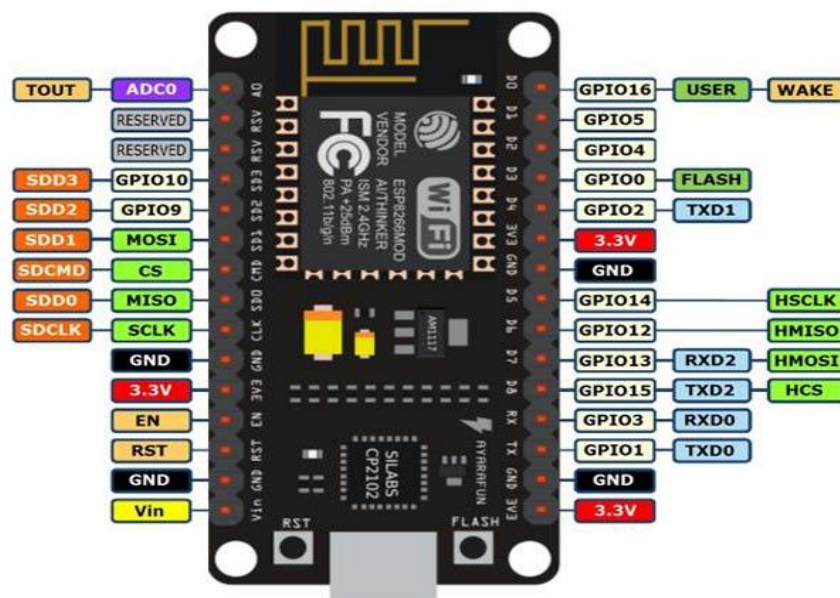


Figure 4: NodeMCU Pins

GPIO

The General-Purpose Input/Output Interface is referred to as GPIO. The 17 GPIO pins of the ESP8266EX can be configured for a variety of purposes using the appropriate registers. Information is kept in computer registers when a GPIO is configured as a source of input. Additionally, the input may be configured to produce level- or edge-triggered CPU interruptions. Internal pull-up, pull-down, high impedance, or both can be selected for each GPIO. The input and output buffers on the non-inverting, Trista and bidirectional IO pads have tristate control inputs. Additional functions like IR remote control, I2C, I2S, UART, PWM, etc. can be multiplexed onto these ports.

SDIO

SDIO stands for Safe Digital I/P or O/P Interface. The ESP8266EX has one Slave SDIO, and its specifications are given below. It supports both the 4-bit 50 MHz and 4-bit 25 MHz SDIO variants.

Pin Name	Pin Num	IO	Function Name
SDIO_CLK	21	IO6	SDIO_CLK
SDIO_DATA0	22	IO7	SDIO_DATA0
SDIO_DATA1	23	IO8	SDIO_DATA1
SDIO_DATA_2	18	IO9	SDIO_DATA_2
SDIO_DATA_3	19	IO10	SDIO_DATA_3
SDIO_CMD	20	IO11	SDIO_CMD

Figure 5: Pin Definitions of SDIO

SPI/HSPI

SPI/HSPI stands for Serial Peripheral Interface.

There are three SPIs in an ESP8266EX.

One uniform Slave/Master SPI

Unique SDIO/SPI

A solitary, common Slave/Master HSPI

Hardware can perform each of these pins' functions. Below is a description of the pin definitions.

General SPI (Master/Slave)

Pin Name	Pin Num	IO	Function Name
SDIO_CLK	21	IO6	SPICLK
SDIO_DATA0	22	IO7	SPIQ/MISO
SDIO_DATA1	23	IO8	SPID/MOSI
SDIO_DATA_2	18	IO9	SPIHD
SDIO_DATA_3	19	IO10	SPIWP
U0TXD	26	IO1	SPICS1
GPIO0	15	IO0	SPICS2

Figure 6: Pin Definitions of SPI's

I2C Interface

The ESP8266EX features a single I2C port for establishing communications with external devices like sensors and microcontrollers. The I2C pin definition is as follows.

Pin Name	Pin Num	IO	Function Name
MTMS	9	IO14	I2C_SCL
GPIO2	14	IO2	I2C_SDA

Figure 7: Pin Definitions of I2C

It is possible to use the two distinct I2C Master and Slave protocols. I2C interface features may be implemented via software programming, and the highest possible frequency of the clock is 100 kHz. The I2C clock frequency should, in general, be higher than the slave device's weakest clock frequency.

Universal Asynchronous Receiver Transmitter (UART)

The following definitions apply to the two UART ports on the ESP8266EX: UART0 and UART.

Pin Type	Pin Name	Pin Num	IO	Function Name
UART0	U0RXD	25	IO3	U0RXD
	U0TXD	26	IO1	U0TXD
	MTDO	13	IO15	U0RTS
	MTCK	12	IO13	U0CTS
UART1	GPIO2	14	IO2	U1TXD
	SD_D1	23	IO8	U1RXD

Figure 8: Pin Definitions of UART

Hardware implementations for data transfers to and from UART interfaces are possible. UART interfaces may transmit data at a rate of up to 115200 x 40 (4.5 Mbps). It is possible to communicate using UART0. It encourages flexible control. UART1 is typically used to print logs because it only has a data send signal (Tx).

PWM

PWM stands for Pulse-Width Modulation. There are four PWM output ports on the ESP8266EX. Users can expand them themselves. The following is a list of the pin definitions for PWM interfaces.

Pin Name	Pin Num	IO	Function Name
MTDI	10	IO12	PWM0
MTDO	13	IO15	PWM1
MTMS	9	IO14	PWM2
GPIO4	16	IO4	PWM3

Figure 9: Pin Definitions of PWM

Software programming can be used to implement PWM interface capabilities. For example, the PWM operation is accomplished in the LED smart light demonstration by interrupting the timer, and up to 44 ns is the lowest resolution. The frequency ranges of programmable PWM range from 1000 to 10000, or 100 to 1×10^3 Hz. The duty ratio is 1/22727 when the PWM frequency is 1×10^3 Hz, and the refresh rate is 1 kHz, over 14-bit resolution, is attained.

Functional Overview

To connect with SPI hosts on other processors, this protocol makes use of the ESP8266's SDIO mode. The electrical interface is connected to the SCLK, MOSI, and MISO, and interrupt signal number one of the SPI protocols through signal line number 4 (note that there is no CS signal). There may be some differences between downloading the ESP8266 SDIO and downloading other programs. The shared pin between When the ESP8266 is powered on, the operating system instantly reads both the SPI interface and the SDIO interface. Therefore, it is suggested that you use the SDIO module communication protocol. To connect with SPI hosts on other CPUs, this protocol makes use of the ESP8266's SDIO mode. Signal line number 4, which broadcasts the SPI protocol's Serial Clock Signals, Master Out Slave In, Midcontinent Independent System Operator, and interrupt signal number 1 (notice that there is no CS signal), connects the electrical interface. Downloading the ESP8266 SDIO might not be the same as downloading other programs. Therefore, it is recommended to use the SDIO module connection protocol.

SOFTWARE TIPS

When using Arduino 0010 to bootload an Atmega8 chip, there is an order (- i800) that delays bootloader execution by 10 minutes. Use the order line instead of IDE by leaving out "- i800" and replacing it with "- F" instead or use Arduino 0007 IDE if you need to utilize the bootloader. The Arduino 0010 does a fantastic job at transferring drawings.

ARDUINO S3v3 NEW FEATURES

- functioning reset pin on the Arduino;
- drove pin 13 of local supply, with a current limiter resistor;
- locally accessible LEDs for TX and RX;
- power was driven with a sufficient current limiter resistor (consumption was less than 20mA);
- auto reset comprises;
- jumper auto reset empower/impairing to prevent unauthorized resets;
- a jumper to prevent "RX skimming blunder" and turn on the RX outer draw-down resistor while disabling sequential communication. When sequential correspondence is not required, this feature enables the usage of computerized pins 0 and 1 as ordinary pins;

- the same board orientation is shared by all comparable components (diodes, semiconductors, LEDs, capacitors), making installations easier to finish and less error-prone;
- Bigger wires, bigger cushions, and more space between wires make drawing, binding, and penetration easier. There are also no short circuits, patching extensions, or open wires.
- All jumpers are the proper point type, allowing for the use of shield boards.

WIFI Module



Figure 10: WIFI Module

Microcontrollers or Wi-Fi modules manage data transmission and reception over Wi-Fi. Additionally, they may reply to Wi-Fi-transmitted commands. The use of Wi-Fi modules allows for device-to-device communication. They are most frequently used on the IOT.

LoRa

A low-power wide area network (LP-WAN) called LoRa was created particularly for the Internet of Things (IoT) and smart sensor applications. LoRa, as its name suggests, is a significant participant in IoT networks since it can communicate across long distances while consuming less power.



LoRa RF click

Figure 11: LoRa

WORKING

A message transmitted by any device can be received by one or more gateways using LoRa technology. The central network will receive the communications and send them there to be processed. All pertinent applications will get these notifications, which will be handled by a sophisticated server architecture.

The LoRa Alliance is an open, non-profit organization with a membership that creates, updates, and enhances the LoRa protocol. One of the main forces behind this extremely effective LP-WAN technology is the Internet of Things.

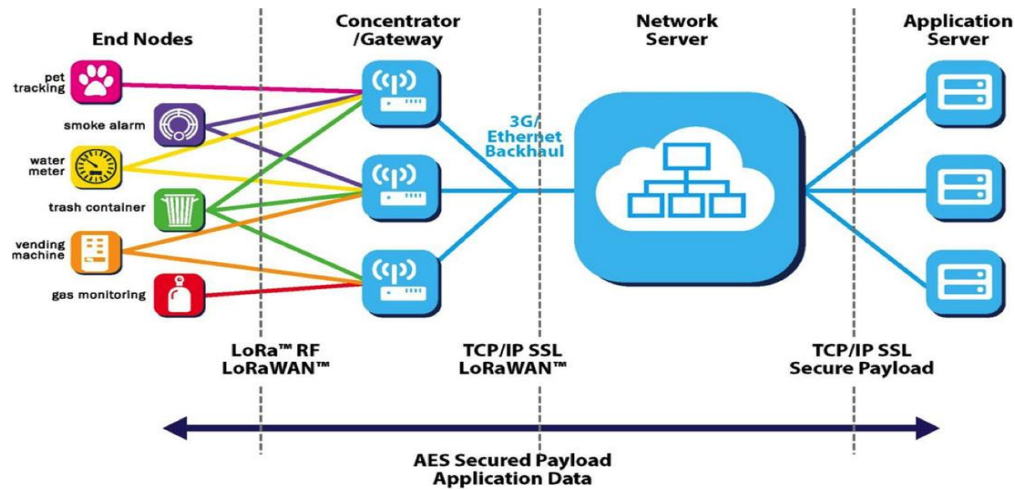


Figure 12: Working of LoRa

Why LoRa®?

LoRa Fills a Technology Gap

IoT has undergone a revolution because of LoRa Technology, which allows for low-power data transmission across large distances. By delivering data packets when linked to a non-cellular Lora WAN network, LoRa devices offer a variety of Internet of Things applications. Lora WAN fills the technological void between cellular and Wi-Fi/BLE-based networks, which are limited by the requirement for large amounts of power or bandwidth or by their inability to reach areas deep inside buildings. LoRa technology can be applied to indoor or outdoor use cases in smart cities, smart buildings, smart homes, smart metering, smart agriculture, and smart supply chain and logistics.

LoRa Enhances Wireless Fidelity, Bluetooth, and Cellular

Like cellular, LoRa Technology is suitable for outdoor applications and is highly secure from end devices to the application server. Lora WAN offers indoor applications and runs in an unlicensed band. When used in public, private, or hybrid networks, LoRa devices and the Lora WAN protocol combine the benefits of Wi-Fi and cellular networks to provide an efficient, flexible, and reasonably priced connectivity option for IoT applications. Simple sensor data can be used to power analytics platforms for artificial intelligence and machine learning. These call for diverse data, which inexpensive LoRa-capable sensors might be able to deliver.

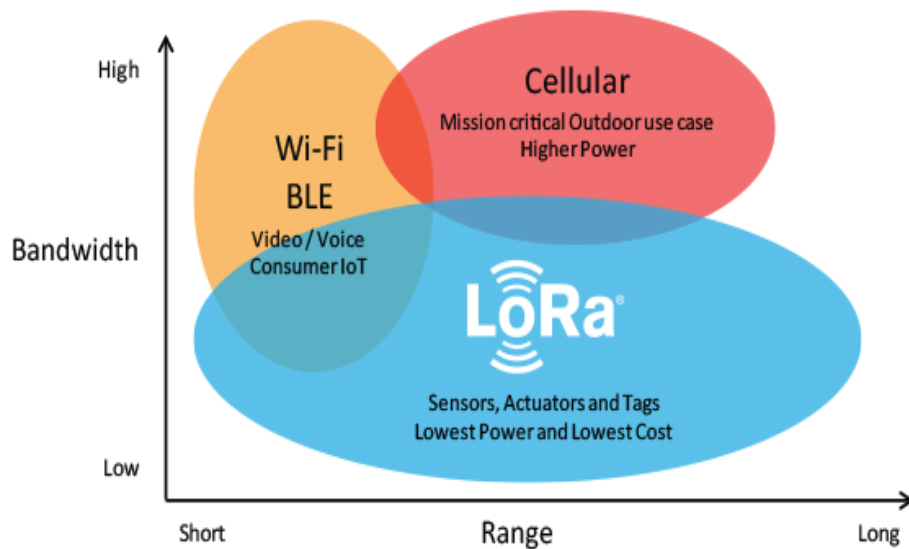


Figure 13: LoRa Complements Wi-Fi, Bluetooth and Cellular

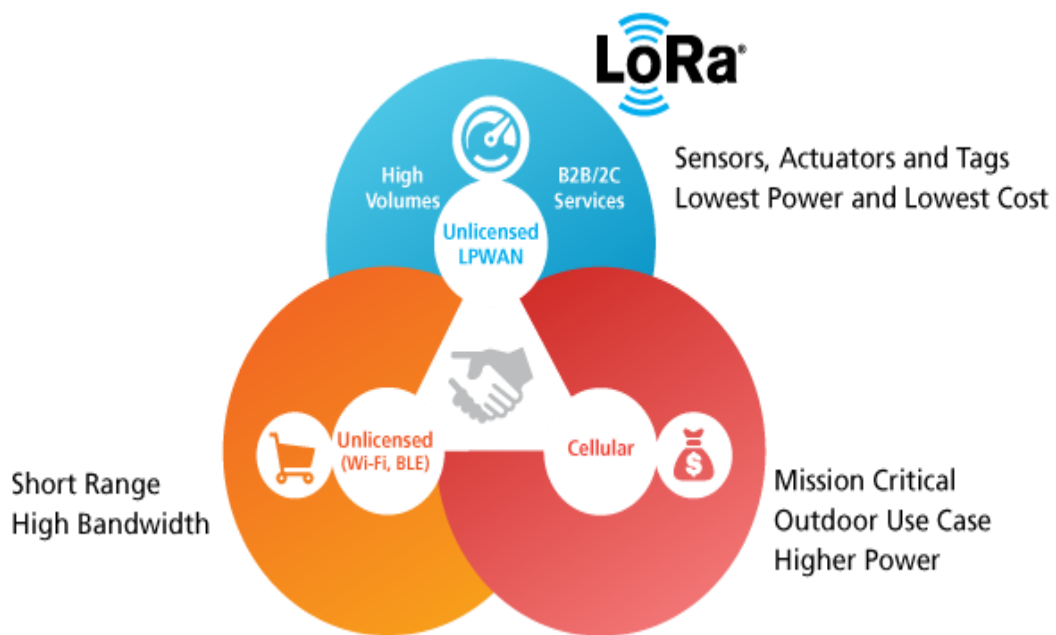


Figure 14: LoRa Availability

Network Availability Near You

The open LoRa WAN open standard, upon which LoRa devices depend, is supported by The LoRa Alliance®, a free, non-profit organisation devoted to encouraging its use internationally. The LoRa Alliance ecosystem, which has more than 500 members and is growing, includes more than 100 public network operators and offers service in 58 countries. Public, private,

and hybrid Lora WAN networks are accessible in more than 100 countries. Implementing solutions quickly is made easy and efficient by the pre-existing infrastructure.

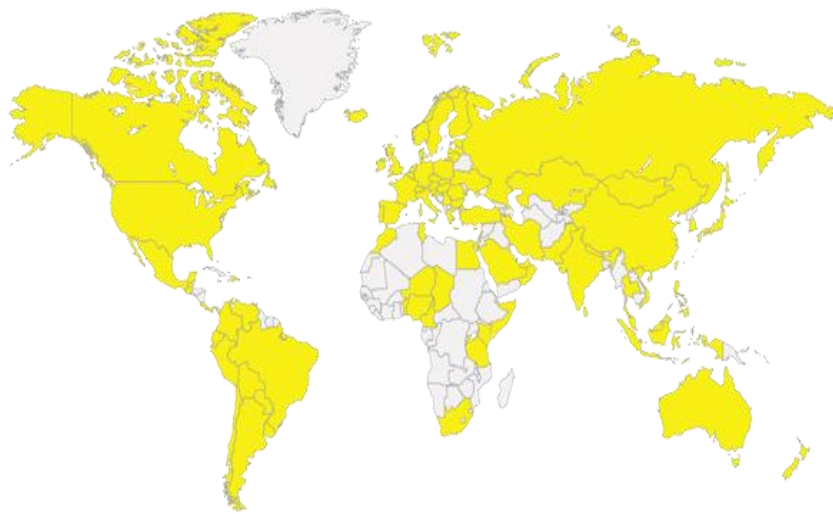


Figure 15: LoRa Network Across the Globe

A Rapidly Scaling Ecosystem

The LoRa and Lora WAN ecosystem, which is crucial to the development of devices, networks, and applications, is made up of a diverse variety of network operators, hardware manufacturers, software developers, service providers, academic institutions, and trade organisations. Thanks to the broad adoption of LoRa technology, the ecosystem is constantly growing and doing so at a surprising rate.



Figure 16: A Rapidly Scaling Ecosystem

Simplified Go-To-Market

Entrepreneurs can now successfully launch IoT solutions despite enduring challenges thanks to Semtech's LoRa Technology. The entire LoRa Technology ecosystem, from silicon to services, allows end-to-end integration. Using an unlicensed band, LoRa Technology provides an open platform that may be customised for a variety of successful business models. The standardised LoRa WAN protocol's interoperability enables the scaling, bundling, and evolution of solutions. The goal of Semtech and its ecosystem partners is to make it simpler for IoT entrepreneurs to market their products and demonstrate useful use cases. A great location for innovators to start is the LoRa Developer Portal, an online community for IoT and M2M developers.

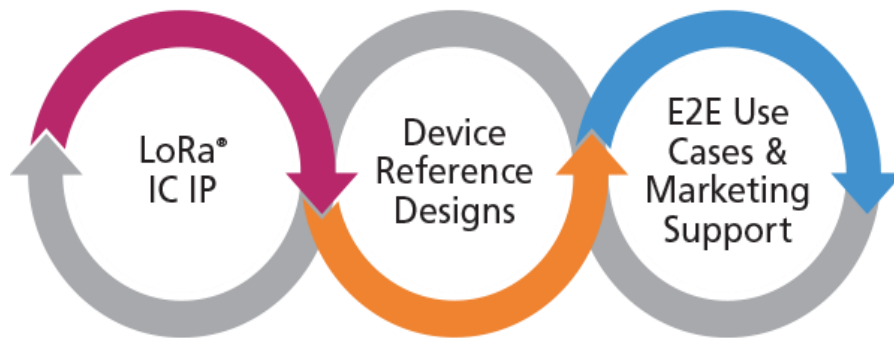


Figure 17: Simplified Go-To-Market

What is a LoRa Gateway?

A sophisticated piece of software created especially for the LoRa network to facilitate communication is known as a LoRa gateway. Between end devices and the application network server, it serves as a smart hub. Every IoT-enabled piece of hardware will eventually come with it preloaded to support a variety of applications.

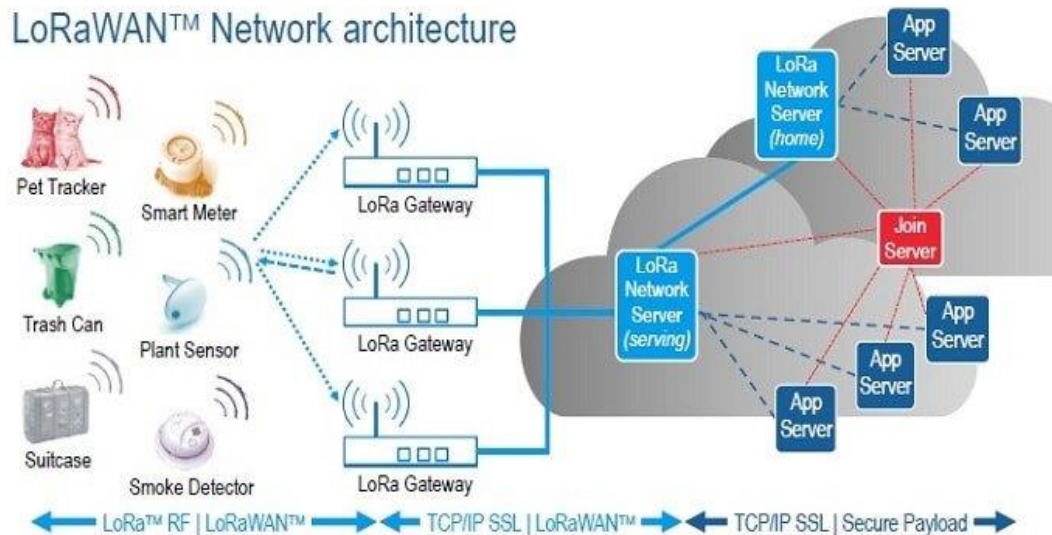


Figure 18: LoRa Architecture

Specifications for LoRa WAN

- The LoRa WAN standard is compliant with IEEE 802.15.4.
- ISM bands of 433, 868, and 915 MHz are licensed-free.
- 125 KHz, 250 KHz, and 500 KHz are the bandwidths.
- Chirp spread spectrum-based modulation is suitable for boosting sensitivity.
- The maximum range is 20 KM, and the highest data rate is 50 kbps.

Benefits of LoRa WAN (LP-WAN difficulties)

Low-cost hardware and unlicensed spectrum contribute to the low implementation costs; Long-range coverage and penetration into buildings; simpler programming; provides a secure communication network; the network that is scalable to allow future upgrades; Access to remote management and control; connectivity and accessibility to cloud services; exceptionally intelligent architecture.

Application and Future of LoRa WAN Technology

Smart City: The following are examples of how IoT and LoRa WAN will be used in "smart cities" in the future: "smart lighting," "air condition and contamination monitoring," "smart

parking and automobile management," "infrastructure and construction management," "fire prediction and administration," and "waste treatment."

Industrial Applications: A wide range of industrial applications are suited for LoRa WAN. Item location and monitoring; Shipping and transport; radioactive and leakage detection; intelligent sensor technologies.

Applications for smart homes: There will be billions of smart home devices and gadgets connected to the internet in the future. Home automation for IoT enables smart appliances and improves home security.

Healthcare: LoRa is one of the greatest options for effectively linking medical equipment. Health monitoring equipment and management.

Agriculture: Applications for smart farming and agriculture can be made with LoRa technology. Monitoring of temperature and moisture levels; smart agricultural and animal management; water level sensors and irrigation control.

Voltage Sensor

This sensor is used to compute, monitor, and figure out the supply of voltage. This sensor can identify the AC or DC electrical voltage level. This sensor may take voltage as input and output switches that are needed, analogue voltage signals, electrical signals, signals from audio, etc. Other sensors are capable of producing signals with AM, PWM, or FM characteristics. Some sensors generate sine or pulse waveforms. The voltage divider could affect the readings made by these sensors.



Figure 19: Voltage Sensor

Electrical or optical impulses can be recognized and responded to by sensors. Current sensor and voltage sensor methodologies have successfully replaced conventional current and voltage monitoring techniques.

The sensor includes an O/P as well as an I/P. The most crucial parts of the input side are the +ve and -ve pins. The device's two pins can be used to link the +ve and -ve pins of the sensor. The +ve and -ve pins of the sensor can be linked to the positive and negative pins of the gadget. Supply voltage (Vcc), ground (GND), and analogue output data are this sensor's primary outputs.

How to Use Voltage Sensor?

An Arduino or other microcontroller can be readily connected to a voltage sensor. The screw terminals of the voltage sensor should be connected to the voltage source's VCC and GND, the voltage of which is being monitored. The Analogue pin and GND of the Arduino should be connected to the Source and - (GND) ports of the voltage sensor, respectively.

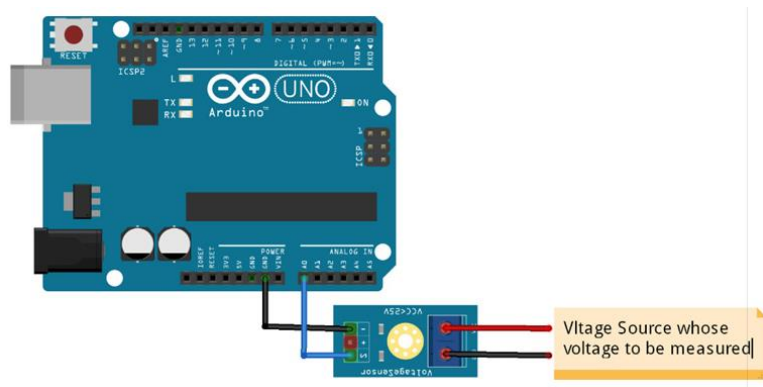


Figure 20: Voltage Source whose voltage is to be measured.

Types of Voltage Sensors

The source of voltage can be located, observed, and measured using a voltage sensor. It has the ability to measure voltage levels at both AC and DC levels. The voltage sensor receives voltage as its input, and it can generate outputs that take the form of switches that operate, signals of analogue electrical, signals of the audio, current levels of analogue frequencies, as well as o/p that are frequency controlled.

In other words, while certain voltage sensors can produce outputs with amplitude, pulse width, or frequency modulation, Others are limited to creating sine or pulse trains.

Voltage dividers serve as the foundation for the measurement of voltage sensors. Voltage sensors of the resistive and capacitive types are the two most popular types of voltage sensors.

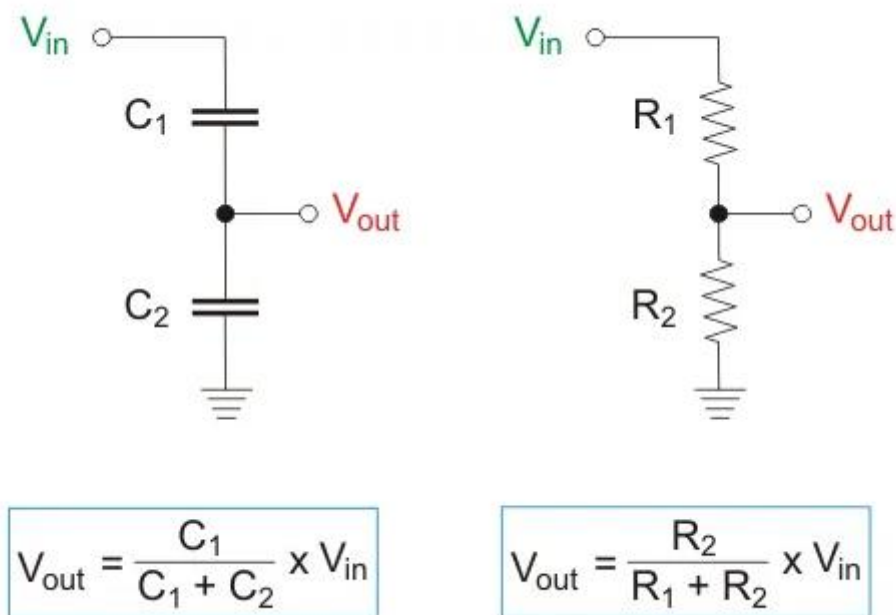


Figure 21: Capacitive Type Voltage Sensor and Resistive Type Voltage Sensor.

Capacitive Voltage Sensor

In a capacitor, a non-conductor is sandwiched between two conductors (or two plates), as is well known.

The term "dielectric" refers to a non-conducting substance. Electricity will start to flow after an AC voltage is introduced across these plates because the opposite plate's voltage will act to either attract or repel electrons.

Without any hardware connections, the field between the plates will construct an entire AC circuit. A capacitor functions in this manner.

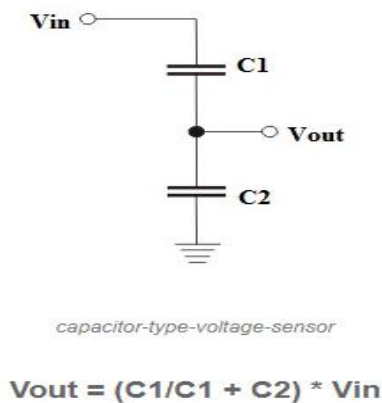


Figure 22: Capacitive Type Voltage Sensor

The voltage distribution between two capacitors that are linked in series can then be discussed. The component with the high impedance often sees a high voltage development in series circuits. Capacitance and impedance (capacitive reactance), in the case of capacitors, usually have reversed proportions.

$$X_C = \frac{1}{2\pi fC}$$

The relation between voltage and capacitance is given as:

$$V = \frac{Q}{C}$$

The letters Q, C, X, and f stand for capacitance, frequency (Hertz), capacitive reactance, and charge (Coulomb).

According to the two connections above, the smallest capacitor will experience the greatest buildup of voltage. This straightforward idea serves as the foundation for capacitor voltage sensors. Consider bringing the sensor's tip close to a live conductor while holding it in your palms.

Here, expanding a series capacitive coupling circuit with a high-impedance sensing component.

The real-time voltage at the tip of the sensor is presently connected to the smaller capacitor. The passive voltage measurement devices that you use within the house operate under the premise that the light or buzzer signal will turn on when the voltage in its entirety rises throughout the measuring circuit, which has the ability to detect voltage.

Voltage Resistive Sensor

There are two techniques to convert the voltage from the resistance of the sensor element. The first and simplest technique is to apply a voltage to the sensor and reference resistor in the below-illustrated resistor divider circuit.

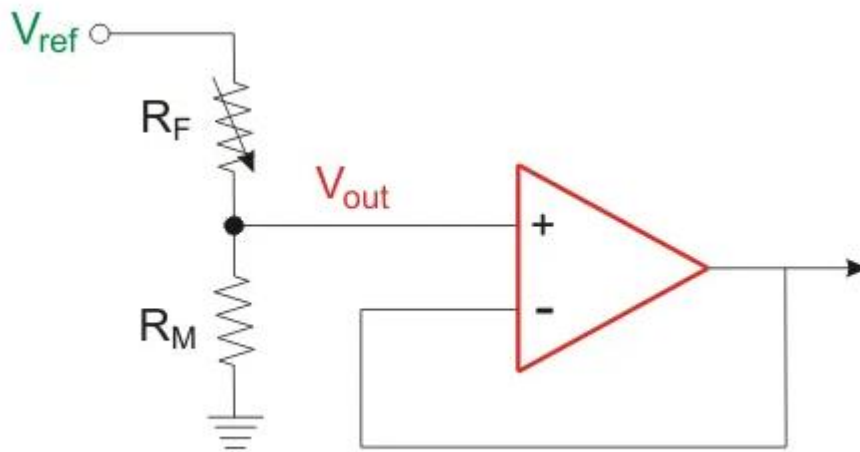


Figure 23: Voltage Resistive Sensor

After being buffered, the voltage produced is sent to the amplifier through a measurement resistor or sensor. The sensor's output voltage can be expressed as

$$V_{out} = \frac{R_M}{R_M + R_F} \times V_{ref}$$

The amplifier that is included in this circuit will magnify all of the voltage generated by the sensor, which is a downside. This is accomplished using the second way, which is seen below and employs a resistance bridge, however, it is desirable to simply amplify the voltage shift brought on by a change in the sensor's resistance.

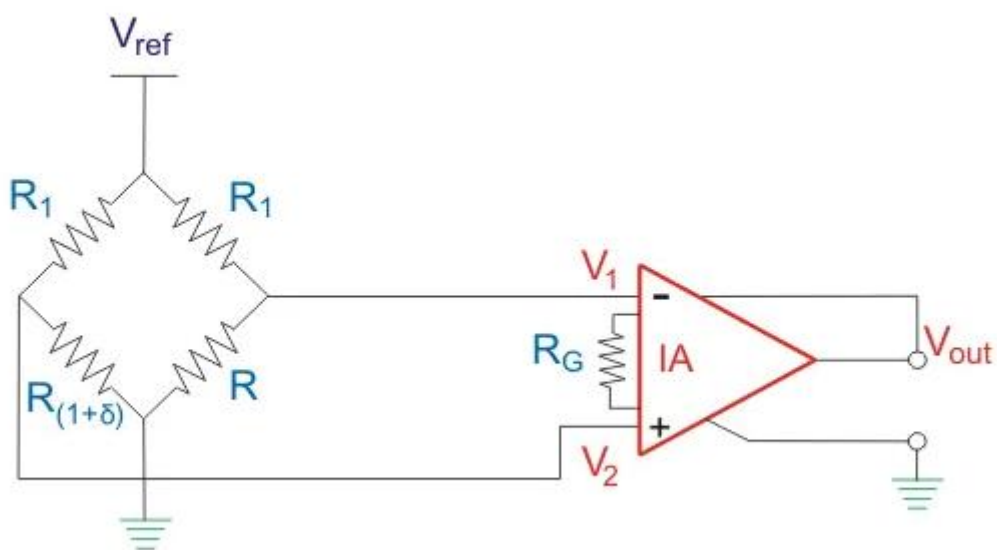


Figure 24: Voltage Resistive Sensor with a resistance bridge

In these instances, an output voltage is

$$V_{out} = A \times \frac{R_M}{R_M + R_F} \times V_{ref} \frac{\delta}{1 + \left(\frac{R}{R_1}\right) (1 + \delta)}$$

When the output voltage approaches $R_1 = R$,

$$V_{out} = \frac{A}{4} V_{ref} \delta$$

‘A’ represents the instrumentation amplifier's gain.

‘δ’ represents the modified sensor's resistance in a way that is similar to physical action.

Configuration of Pinout

Name of the PIN	Description
VCC	The positive terminal of the External voltage source (0-25V)
GND	External voltage source terminal with a negative polarity
S	The Arduino's Analogue pin is connected to the Analogue pin.
+	Not Related
-	Ground Pin connected to GND of Arduino

Advantages of Voltage Sensors

Wide dynamic range, non-saturable, very accurate, small in size and weight, and environmentally beneficial.

Applications of Voltage Sensors

Power consumption minimization; load sensing; security switching; temperature management; and failure of electricity identification.

Features & Specifications

The input Voltage range is from 0 to 25V; the Detection voltage range is from 0.02445 to 25V. It has the following features: 0.00489V analogue voltage resolution, no external components, microcontroller compatibility, small size, low cost, and wide availability. Measures 4 x 3 x 2 cm.

PV Panel

The idea behind PV effects is that when electrons are displaced, an electric current result. The outermost electrons of the atoms in particular semiconductor elements are excited for this to occur, thanks to photons (light particles).



Figure 25: PV Panel

SOFTWARE REQUIREMENTS:

- Arduino IDE
- Programming: Embedded C
- Cloud

Arduino IDE

File

- New

The interface is created from the start with a default picture in place.

- **Open**

With Opened, you can navigate the computer's CDs and directories while importing a sketch file.

- **Open Recent**

Open Recent Shows provides quick access to a selection of the most recent sketches.

- **Sketchbook**

Sketchbook Shows all currently open drawings that are organized in a folder structure in the notebook; selecting the file name displays the sketch in an entirely new editor instance.

- **Examples**

Any example provided by the Arduino Software (IDE) or library shows up in this menu item. All the examples are structured in a tree that allows easy access by topic or library.

- **Close**

Closes the instance of the Arduino Software from which it is clicked.

- **Save**

Saves the sketch with the current name. If the file hasn't been named before, a name will be provided in a "Save as..." window.

- **Save as...**

You can rename the current sketch when you save it by selecting Save as...

- **Setup of Pages**

The printing Page Setup window is displayed.

- **Print**

Print This choice makes the current sketch —as defined in Page Setup—sent to the printer.

- **Preferences**

Opens the Preferences box, where you can modify additional IDE settings and the IDE interface's language.

- **Quit**

The IDE's windows are all closed when it is quit. The subsequent time the IDE is launched, by default, when quit is chosen, the same drawings that were open will be shown.

Edit

- **Undo/Redo**

It reverses one or more editing stages in the past.; you can then redo a step after undoing one.

- **Cut**

Eliminates the text that was selected from the text editor and copies it to the notepad.

- **Copy**

The content that is now chosen in the editor is copied and added to the notepad.

- **Copy for Forum**

Post to Forum copies your sketch's code to the clipboard with syntax colouring for forum use.

- **Copy as HTML**

By selecting "Save as HTML," you can copy the code in HTML for your sketch to the notebook to post it online.

- **Paste**

Inserts the editor's cursor position's current view of the clipboard's contents.

- **Select All**

Mark all the editor's work with the Select All Selects option.

- **Uncomment/Comment**

On each selected line either an additional or removed // comment marker is used.

- **Increase/Decrease Indent**

Boost/Decrease Indenting either pushes the content to be shifted right into a single space or gets rid of the space at the start of each chosen line, depending on your desire.

- **Find**

Use the Find and Replace box, which is displayed after selecting the Find command, to choose how to search for the text The present drawing contains.

- **Find Next**

Highlights the character's subsequent appearance, if any, in relation to the cursor position, as provided as the criteria for the search item in the Find box.

- **Find Previous**

Highlights the previous occurrence - if any - of the string specified as the search item in the Find window relative to the cursor position.

Sketch

- **Verify/Compile**

Verify/Compile does a compilation error check on your sketch and shows details about memory usage for variables and code in the terminal window.

- **Upload**

After being compiled, the binary file is uploaded to the designated board through the designated Port.

- **Upload Using the Programmer**

Utilize a coder to upload. Therefore, the firmware on the board will be changed. Choose Tools > Burn Firmware to replace it and resume downloading to the USB sequential connection. You may still use the complete flash drive for your sketch thanks to this. Attention: This is NOT a command to fire the fuses. Use the Tools -> Burn Bootloader command to accomplish this.

- **Export Compiled Binary**

Creates a .hex document that can be delivered to the device with various tools or archived for later use.

- **Show Sketch Folder**

Shows the currently open drawing folder.

- **Include Library**

By including #include instructions at the beginning of your code, you can add a collection of files to your sketch. See the libraries below for more information. You may also open the Library Manager from this menu choice and add freshly created libraries from .zip files.

- **Add File...**

Creates an original file that will be duplicated from its present location and added to the sketch. In the sketch window, a new tab with the new file opens. Using the tab menu, which is accessed by clicking on the tiny triangle icon next to the serial monitor symbol on the right side of the toolbar, files can be deleted from the sketch.

Tools

- **Auto Format**

Increasing the indentation of the statements inside curly braces neatly styles your code by matching the gap between and ending brackets with curves and spacing the entire thing.

- **Archive Sketch**

Retro Sketch makes a zip file with a duplicate of the most current drawing inside of it. The archive and the drawing may both be found in an identical folder.

- **Reload after making corrections in encoding.**

Ensures that the character map encoding used by an editor is consistent with that used by other OS.

- **Serial Monitor**

Transmission of data begins on any attached board on the currently chosen Port as soon as Serial Observer opens the serial monitor screen. If the board allows Restore via Sequential-Port Opening, doing this frequently resets it.

- **Board**

Choose the board that is presently open. Listed below is a list of each board's descriptions.

- **Port**

This menu contains a list of all serial devices on your computer, real or virtual. The top-level tools menu should immediately refresh each time you use it.

- **Programmer**

A hardware programmer can be used to program a board or chip in place of the built-in USB-serial connection. In most cases, you aren't going to need this, but if transferring an operating system to a new microcontroller, you will.

- **Burn Bootloader**

The Arduino board's microcontroller can be burned with a bootloader using the items in this menu. This is normally not necessary when using an Arduino or Genuino board, but it could be helpful if you purchase a new ATmega microcontroller—which commonly does not arrive with a bootloader. Verify that the correct board was selected from the Boards option before burning the bootloader onto the target board. This command also correctly sets the fuses.

Help

This website provides access to a variety of documentation that comes with the Arduino Software (IDE). You may still view the section on Getting Started, Reference, as well as this Integrated Development Environment reference manual, and other materials locally even if you don't have an internet connection. The newspapers may include links to our website as well as multilingual versions of their online editions.

- **Find in Reference**

The single interactive option in the Help menu chooses the appropriate page for the function or command now being used from the local copy of the Reference.

Sketchbook

In the Arduino Software (IDE), a sketchbook is referred to as a common place to save programming (or drawings). By selecting Files > Workbook from the available options or by pressing the Open link on the toolbar, you may access the drawings in your sketchbook. When you initially open the Arduino program, it will begin generating a directory that contains your sketchbook. You may check or alter the location of the sketchbook using the Preferences box. The .ino file extension is used to store files as of version 1.0. For earlier generations, use the .pde extension. Versions with a value of up to 2.0 continue to allow you to open .pde files, but they now come with the .ino extension by default.

Numerous Files, Compilations and Tabs

Manages drawings that contain several files, each with its tab. These can be C++ files (.cpp), c files with the .c extension, header files (.h), and more. Ordinary Arduino code files (no apparent extension), or any other kind of file.

Uploading

Before submitting your sketch, you must choose the relevant choices through the Tools > the Board and Equipment > Ports menus. The boards are described below. The serial port names on the Mac are probably /dev/tty.usbmodem241, /dev/tty.usbserial-1B1, or /dev/tty.USA19QW1b1P1.1 (for a serial board paired with a Keyspan USB-to-Serial converter) for an Uno, Mega2560, or Leonardo. To learn more, look up USB serial devices in the Windows Device Manager's ports section. For a USB board, the most likely COM ports on Windows are COM4, COM5, or COM7 or above. A serial board has two options: COM1 or COM2. On Linux, it ought to be /dev/ttyACMx, /dev/ttyUSBx, or something of the like. Once you've chosen the proper serial port and board, click the "Upload" button in the taskbar

or select Upload through the sketch menu. The upload will start after the current Arduino boards have been automatically reset. On earlier (pre-Diecimila) boards with auto-reset, you must press the reset button before starting the upload. The TX and RX LEDs on the majority of boards will begin to flash immediately after the design is uploaded. The Arduino Software, also known as the IDE, will provide a notification and an error when the upload is complete.

To upload drawings, utilize the Arduino bootloader, a small application put on the board's microcontroller. It makes it possible to upload code without using additional hardware. Just before the processing of the most recent sketch transferred to the microcontroller begins whenever the circuit board resets itself, the bootloader is briefly active. When the pin 13 onboard LED is activated, it will blink. bootloader launches, or when the board is reset.

Libraries

More functionality is now available for drawings, including the ability to modify data or interact with hardware. To use the library in the drawing, choose it from the drawing's Import Library option. By using one or more `#include` statements, you may finish off your drawing and integrate it with the library. Libraries require extra room because they are included in your sketch that is submitted to the board. If a sketch is no longer necessary, just delete the `#include` declarations at the leading portion of your code.

There is a list of bookstores in the reference. A few libraries are included with Arduino programming. Other people can download them via the Resources Administrator at a few other locations. You can utilize a collection of files you've exported from a zip folder in an open drawing using IDE version 1.0.5. For information on how to install a third-party library, go to these instructions.

View this tutorial to find out how to create your library.

Third-Party Hardware

To your sketchbook directory's hardware directory, you may add support for external hardware. Board definitions, core libraries, bootloaders, and programmer definitions are a few examples of platforms that can be added there and displayed in the board menu. Create the computer's hardware directory first, then, after installation, extract the software from the third-party different platforms into a different sub-directory. (Avoid giving the sub-directory the name "Arduino"; doing so will override the Arduino platform.)

It may be uninstalled by removing its directory.

For additional information on how to construct packages for third-party hardware, see the IDE for Arduino 1.5 Third-Party Hardware standard.

Serial Monitor

Displays the serial data that the Arduino or Genuino board, which is a sequential or USB board, is sending. To transmit the text to the board, type it in and press the button marked "send" or enter. Choose the baud rate that corresponds to the rate that Serial was given from the drop-down selection. Start with the illustration. Remember that the programmed Arduino as well as Genuino boards will reset (restart your code execution) when you first connected to the serial monitor interface on Mac, Windows, or Linux.

For more information, see the interfacing page. You may also communicate with the board using Processing, Flash, MaxMSP, etc.

Preferences

You may modify a few options in the preferences dialogue, which is accessible through the File or Arduino menus on Linux and Windows, respectively. The remaining information is located in the configuration file, and its location is specified in the preference dialogue.

Language Support

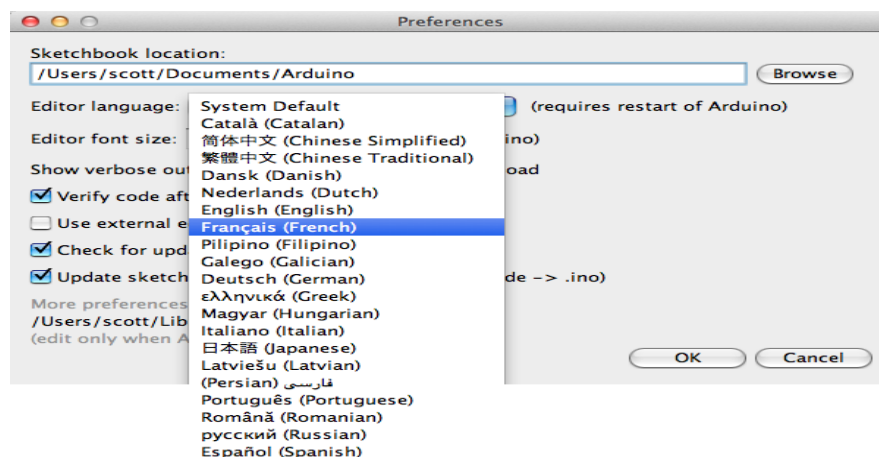


Figure 26: Language Support

Since version 1.0.1, the software that comes with the Arduino (Integrated Development Environment) has been extensively transcribed into over thirty different languages. The

language that starts up in the IDE automatically is selected by the operating system. Note that instead of determining the language used to display the operating system, the language's locale selection, which controls money and date formats, determines this on Windows and perhaps Linux.

Open the Preferences panel after starting the Arduino Software (IDE) if you want to manually change the language. A drop-down box with the accepted languages is located next to the Editor Language. Restart the app after choosing the language you want to use from the menu. The Arduino Software (IDE) will not function if the operating system language you are using is not supported and will use English by default.

The Editor's Languages drop-down list allows you to direct the programme to use the operating system-specific language by choosing System Default. This option will become active once the Arduino Integrated Development Software (IDE) is restarted. Restarting the Arduino Software (IDE), which is necessary, much like your operating system must be restarted after changing its default language.

CHAPTER-4

METHODOLOGY

A monitoring system for the system's renewable energy-producing units is in this post. The many nodes in this system are connected by means of LoRa communication. The power generated is monitored using voltage as well as current sensors, and the information is transferred to a distant collecting node for additional analysis. IoT makes remote data access possible in this case. The project is being developed on the Arduino platform. In this article, any type of embedded controller may be utilized, and may even use the Arduino programming platform. Since Arduino is an open-source platform, funding the project makes more sense. Indoor LoRa devices may still be unable to wirelessly connect with a surrounding GW due to barriers between sensors that can weaken wireless signal strength, result in data losses, and disrupt communication.

4.1 Block Diagram

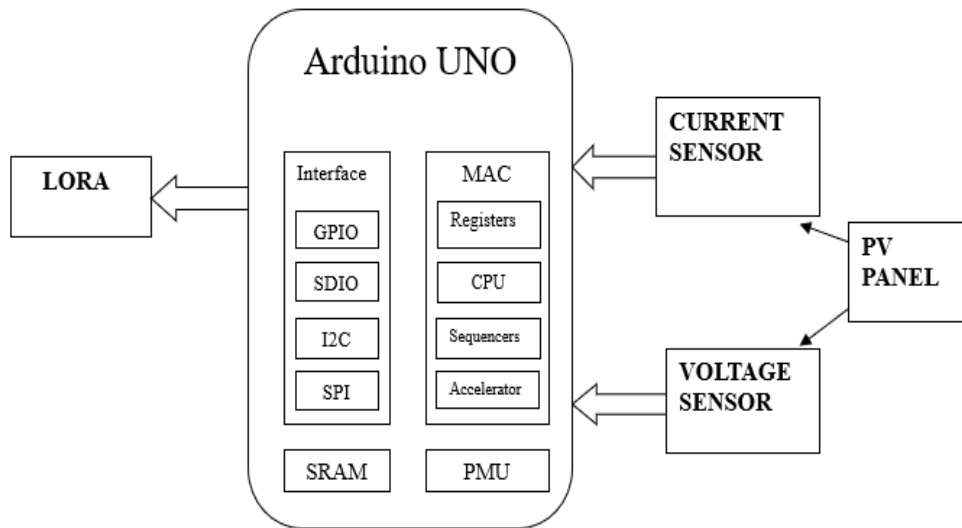


Figure 27: TRANSMITTER (Tx) (SENSOR NODE)

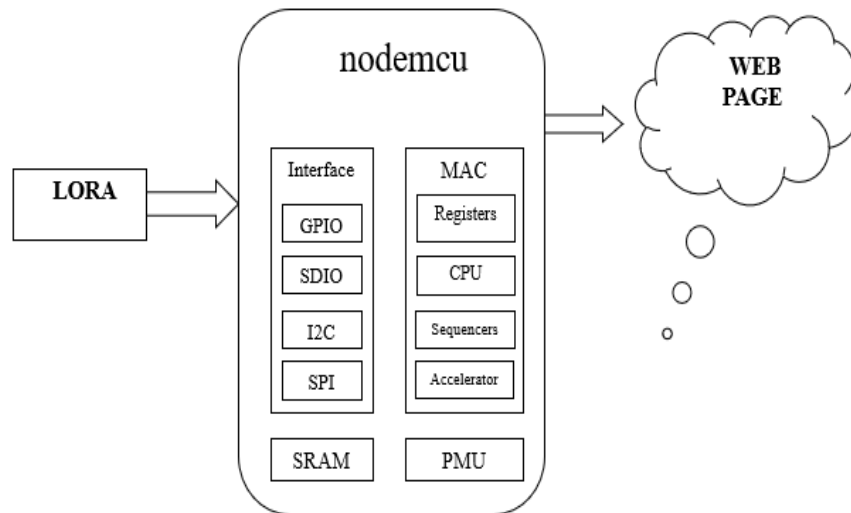


Figure 28: RECEIVER (Rx) (DATA NODE)

Here In This Article, Arduino Uno is used in this sensor node. The analogue interface on Arduino Uno is used to connect both sensors (ADC). The serial interface is used to link LoRa devices. Because it is needed to link this node to the cloud, the data node uses the NodeMCU, which has built-in WIFI. This NodeMCU is connected to a receiver using LoRa.

In the Transmitter node, Electricity comes from the PV Panel. Voltage and current sensor sense how much amount of electricity is stored in the PV Panel. The sensor sends the signal to the LoRa TX through Arduino. In the receiver node, LoRa RX receives the signal from the LoRa TX. LoRa RX sends the signal to the IOT Platform through Arduino.

CHAPTER-5

RESULTS

5.1 Results Discussion

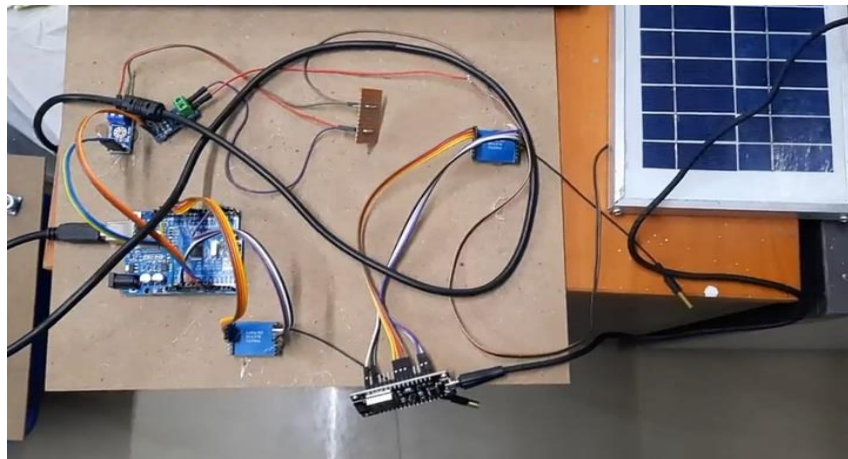


Figure 29: Circuit Diagram

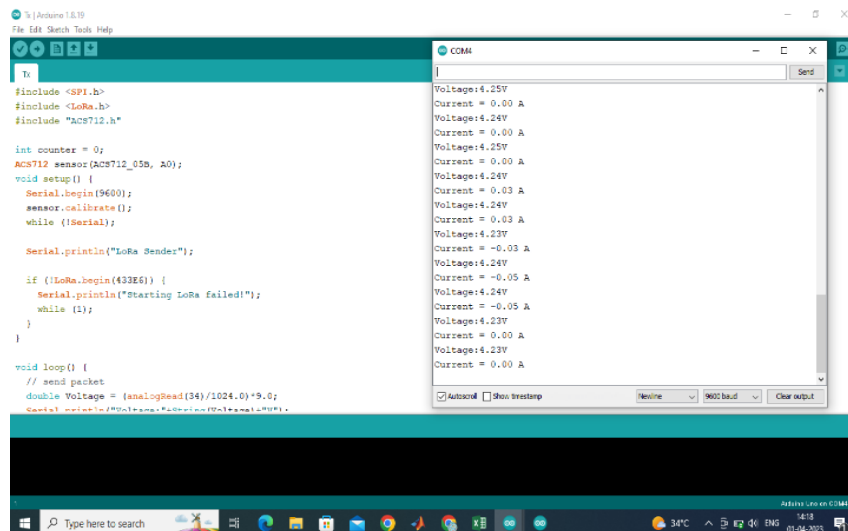


Figure 30: The Output of TRANSMITTER (Tx)-SENSOR NODE

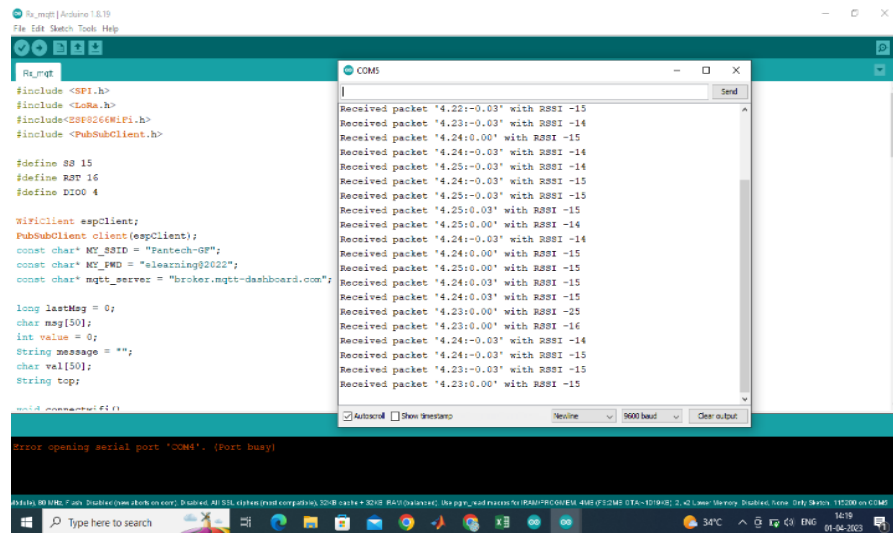


Figure 31: The Output of RECEIVER (Rx)- (DATA NODE)

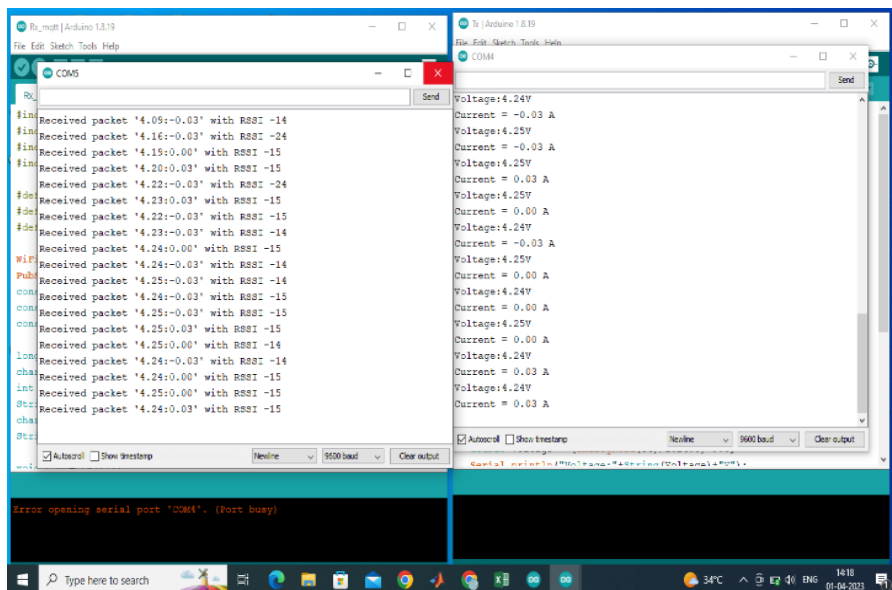


Figure 32: Output

CHAPTER-6

CONCLUSION & FUTURE SCOPE

6.1 CONCLUSION

Further reviews of various studies pertaining to the use of IoT technology in energy management systems have been undertaken in this research. IoT technology has been used in several industries, including smart homes, smart buildings, smart meters, smart environments, and commercial. The cost of the microcontrollers that can be used to create Internet of Things devices varies as well. Because they are affordable and come with tiny modules, including Wi-Fi modules that are installed directly on NodeMCU, Arduino and NodeMCU are the microcontrollers that are most frequently utilized in the construction of IoT-based systems. Since the implementation of IoT demands that each component be ready, such as the availability of devices and the environment/activities, many of its applications in the research environment are still in the simulation/testing phase.

6.2 FUTURE SCOPE

In the future solar power will account for a bigger share of the global generation of power. A rated overall energy should be a Solar platform manufactured. So, it is needed to interface more sensors into this project to make this project more efficient and to increase power production.

REFERENCES

1. Kebotogetse, Otisitswe, Ravi Samikannu, and Abid Yahya. "A concealed based approach for secure transmission in advanced metering infrastructure." *IEEE Access* 10 (2022): 84809-84817.
2. Kumari, Preti, et al. "An energy-efficient smart metering system using edge computing in LoRa network." *IEEE Transactions on Sustainable Computing* 7.4 (2021): 786-798.
3. Alkawsi, Gamal Abdalnaser, and Yahia Baashar. "An empirical study of the acceptance of IoT-based smart meter in Malaysia: The effect of electricity-saving knowledge and environmental awareness." *IEEE Access* 8 (2020): 42794-42804.
4. Unal, Fatih, et al. "Big data-driven detection of false data injection attacks in smart meters." *IEEE Access* 9 (2021): 144313-144326.
5. Kane, Luke, et al. "Network Architecture and Authentication Scheme for LoRa 2.4 GHz Smart Homes." *IEEE Access* 10 (2022): 93212-93230.
6. Liu, Fangxing, Chengbin Liang, and Qing He. "Remote malfunctioned smart meter detection in an edge computing environment." *IEEE Access* 8 (2020): 67436-67443.
7. Xia, Zhuoqun, et al. "Secure session key management scheme for a meter-reading system based on LoRa technology." *IEEE Access* 6 (2018): 75015-75024.
8. Kim, Jin Young, et al. "Detection for non-technical loss by smart energy theft with intermediate monitor meter in smart grid." *IEEE Access* 7 (2019): 129043-129053.
9. Sănduleac, Mihai, et al. "High reporting rate smart metering data for enhanced grid monitoring and services for energy communities." *IEEE Transactions on Industrial Informatics* 18.6 (2021): 4039-4048.
10. Gallardo, José Luis, Mohamed A. Ahmed, and Nicolás Jara. "Clustering algorithm-based network planning for advanced metering infrastructure in smart grid." *IEEE Access* 9 (2021): 48992-49006.

11. Chekired, Djabir Abdeldjalil, Lyes Khoukhi, and Hussein T. Mouftah. "Fog-based distributed intrusion detection system against false metering attacks in smart grid." *ICC 2019-2019 IEEE International Conference on Communications (ICC)*. IEEE, 2019.
12. Badr, Mahmoud M., et al. "Detection of false-reading attacks in smart grid net-metering system." *IEEE Internet of Things Journal* 9.2 (2021): 1386-1401.
13. Liu, Xiaoxue, et al. "A collaborative intrusion detection mechanism against false data injection attack in advanced metering infrastructure." *IEEE Transactions on Smart Grid* 6.5 (2015): 2435-2443.
14. Zanetti, Marcelo, et al. "A tunable fraud detection system for advanced metering infrastructure using short-lived patterns." *IEEE Transactions on Smart grid* 10.1 (2017): 830-840.
15. Li, Beibei, Rongxing Lu, and Gaoxi Xiao. "HMM-based fast detection of false data injections in advanced metering infrastructure." *GLOBECOM 2017-2017 IEEE Global Communications Conference*. IEEE, 2017.
16. Marahatta, Anup, et al. "Evaluation of a lora mesh network for smart metering in rural locations." *Electronics* 10.6 (2021): 751.
17. Cheng, Yao, et al. "Secure smart metering based on LoRa technology." *2018 IEEE 4th International Conference on Identity, Security, and Behavior Analysis (ISBA)*. IEEE, 2018.
18. Wibisono, Gunawan, et al. "Development of advanced metering infrastructure based on LoRa WAN in PLN Bali toward Bali Eco smart grid." *2017 Saudi Arabia Smart Grid (SASG)*. IEEE, 2017.
19. Bagariang, Yosia, Muhammad Imam Nashiruddin, and Nachwan Mufti Adriansyah. "Lora-based iot network planning for advanced metering infrastructure in urban, suburban and rural scenario." *2019 International Seminar on Research of Information Technology and Intelligent Systems (ISRITI)*. IEEE, 2019.
20. Varsier, Nadège, and Jean Schwoerer. "Capacity limits of LoRaWAN technology for smart metering applications." *2017 IEEE international conference on communications (ICC)*. IEEE, 2017.

APPENDIX

CODING

TX:

```
#include <SPI.h>
#include <LoRa.h>
#include "ACS712.h"

int counter = 0;
ACS712 sensor(ACS712_05B, A0);
void setup() {
  Serial.begin(9600);
  sensor.calibrate();
  while (!Serial);

  Serial.println("LoRa Sender");

  if (!LoRa.begin(433E6)) {
    Serial.println("Starting LoRa failed!");
    while (1);
  }
}

void loop() {
  // send packet
  double Voltage = (analogRead(34)/1024.0)*9.0;
  Serial.println("Voltage:"+String(Voltage)+"V");
  double I = sensor.getCurrentDC();
  Serial.println(String ("Current = ") + I + " A");
  String data = String (Voltage)+":"+String(I);
  // Serial.println()
  LoRa.beginPacket();
  LoRa.print(data);
  LoRa.endPacket();

  counter++;

  delay (3000);
}
```

RX:

```
#include <SPI.h>
```

```
#include <LoRa.h>
```

```
#include<ESP8266WiFi.h>
```

```
#include <PubSubClient.h>
```

```
#define SS 15
```

```
#define RST 16
```

```
#define DIO0 4
```

```
WiFiClient espClient;
```

```
PubSubClient client(espClient);
```

```
const char* MY_SSID = "Pantech-GF";
```

```
const char* MY_PWD = "elearning@2022";
```

```
const char* mqtt_server = "broker.mqtt-dashboard.com";
```

```
long lastMsg = 0;
```

```
char msg[50];
```

```
int value = 0;
```

```
String message = "";
```

```
char val[50];
```

```
String top;
```

```
void connectwifi()
```

```
{
```

```
  Serial.println("connection to" + *MY_SSID);
```

```
  WiFi.begin(MY_SSID,MY_PWD);
```

```
  while (WiFi.status()!=WL_CONNECTED)
```

```
  {
```

```
    delay(500);
```

```
    Serial.print(".");
```

```
  }
```

```
  Serial.println("");
```

```
  Serial.println("WIFI CONNECTED");
```

```

    Serial.println("");
}

void callback(char* topic, byte* payload, unsigned int length) {
    top = topic;
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }
    message += '\0';
    // Serial.println(message);
    // Serial.println();
}

void reconnect() {
    while (!client.connected()) {
        Serial.print("Attempting MQTT connection...");
        String clientId = "ESP8266Client-";
        clientId += String(random(0xffff), HEX);
        if (client.connect(clientId.c_str())) {
            Serial.println("connected");
            //      client.publish("outTopic", "hello world");
            // client.subscribe("input");
        }
        else {
            Serial.print("failed, rc=");
            Serial.print(client.state());
            Serial.println(" try again in 5 seconds");
            delay(5000);
        }
    }
}

void setup() {

```

```

Serial.begin(9600);
connectwifi();
client.setServer(mqtt_server, 1883);
while (!Serial);

Serial.println("LoRa Receiver");
LoRa.setPins(SS, RST, DIO0);
if (!LoRa.begin(433E6)) {
    Serial.println("Starting LoRa failed!");
    while (1);
}
}

void loop() {
    // try to parse packet
    client.setCallback(callback);
    if (!client.connected()) {
        reconnect();
    }
    client.loop();
    String data;
    int packetSize = LoRa.parsePacket();
    if (packetSize) {
        // received a packet
        Serial.print("Received packet ");

        // read packet
        while (LoRa.available()) {
            data+=(char)LoRa.read();
        }
        Serial.print(data);

        // print RSSI of packet
        Serial.print(" with RSSI ");
    }
}

```

```
    Serial.println(LoRa.packetRssi());  
  }  
  data.toCharArray(msg, data.length());  
  client.publish("node",msg);  
  // delay(4000);  
}
```


ORIGINALITY REPORT

7%	3%	1%	4%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	grietinfo.in Internet Source	2%
2	Submitted to Coventry University Student Paper	1%
3	Submitted to Durban University of Technology Student Paper	1%
4	arduino.cc Internet Source	<1%
5	Submitted to Visvesvaraya Technological University, Belagavi Student Paper	<1%
6	docplayer.net Internet Source	<1%
7	Submitted to Sreenidhi International School Student Paper	<1%
8	Submitted to SRM University Student Paper	<1%